

LC 310: Minimum Height Trees

Authored by Arpit Raj, LNMIIT Jaipur • July 31, 2026

Target Companies: PhonePe, Citadel, Amazon

Problem Summary: Connections of a tree are given, need to make a valid Minimum Height Tree (MHT) and then return the node label.

Approach 1: Brute Force (BFS/DFS)

- For every node, assume it is the root.
- Run BFS/DFS to compute the maximum depth (height).
- Keep track of the minimum height. *Jiska ans min height aaya wo ans me push back.*
- **Time Complexity:** $O(V^2)$ (Expected TLE).

```

class Solution {
public:
    int nn;
    int bfs(int src, vector<vector<int>>& adj) {
        queue<int> q;
        vector<int> seen(nn, 0);
        q.push(src);
        seen[src] = 1;

        int h = 0;
        while (!q.empty()) {
            int currsz = q.size(); // height nikaalne me level by level
processing
            while (currsz--) {
                int currn = q.front();
                q.pop();

                for (auto &it : adj[currn]) {
                    if (!seen[it]) {
                        seen[it] = 1;
                        q.push(it);
                    }
                }
            }
            h++;
        }
        return h;
    }

    vector<int> findMinHeightTrees(int n, vector<vector<int>>& edges) {
        // approach1
        // ig tle dega
        // har node se bfs run krenge
        // jiska ans min height aaya wo ans me push back

        vector<vector<int>> adj(n);
        nn = n;
        for (auto &it : edges) {
            int u = it[0];
            int v = it[1];

            adj[u].push_back(v);
            adj[v].push_back(u);
        }
    }
}

```

```

int minh = INT_MAX;
vector<int> ans;
for (int i = 0; i < n; i++) {
    int currh = bfs(i, adj);
    if (currh < minh) { // achieved a new low in life
        ans.clear();
        ans.push_back(i);
        minh = currh;
    }
    else if (currh == minh) {
        ans.push_back(i);
    }
}
return ans;
}
};

```

Approach 2: Reverse Pruning (Optimal) ✨

Ek Badi Tagdi Cheez Observe Ki:

- Leave node to root ho nhi skta.
- Baaki middle nodes ko agar root bnaye to height kabhi min nhi hoti.
- To pyaaz ki trh neeche se cheelte jaayenge (Keep removing leaf nodes).
- Ye tab tak krenge jab tak 1 ya 2 node nhi bach jaaye.
- **Centroid can be 1 or 2 (top layer).**

```

class Solution {
public:
    vector<int> findMinHeightTrees(int n, vector<vector<int>>& edges) {
        // har node se bfs krne pe tle aa rha
        // ek badi tagdi cheez observe ki
        // leave node to root ho nhi skta
        // baaki middle nodes ko agar root bnaye to height kabhi min nhi hoti
        // to pyaaz ki trh neeche se cheelte jaayenge
        // ye tab tak krenge jab tak 1 ya 2 node nhi bach jaaye
        // centroid can be 1 or 2 (top layer)

        if(n == 1) return {0}; // edging

        vector<vector<int>> adj(n);
        vector<int> degree(n, 0);

        for(auto &it : edges) {
            int u = it[0];
            int v = it[1];

            adj[u].push_back(v);
            adj[v].push_back(u);
            degree[u]++;
            degree[v]++;
        }

        // leaf nodes remove krte jaayenge
        queue<int> q;

        int rem = n; // last me 1 ya 2 node bacha ke rkhne he
        for(int i = 0; i < n; i++) {
            if(degree[i] == 1) {
                q.push(i);
            }
        }

        while(rem > 2) {
            int currrsize = q.size();
            rem -= currrsize;

            while(currrsize--) {
                int curr = q.front();
                q.pop();
            }
        }
    }
};

```

```

        for(auto &it : adj[curr]) {
            degree[it]--;
            if(degree[it] == 1) {
                q.push(it);
            }
        }
    }

    // last me q me jo bache wo ans
    vector<int> ans;
    while(!q.empty()) {
        ans.push_back(q.front());
        q.pop();
    }

    return ans;
}

};

```